Programmation Orientée Objet (POO) à destination de la robotique

Travaux Pratique : Simulation réaliste

Enseignant : Adam GOUGUET
Mail : adam.gouguet@imt-nord-europe.fr
Année : 2025–2026

Table des matières

1 Objectifs du TP **1**

2 Contexte **1**

3 Simulation réaliste d'un moteur différentiel **1**

3.1 Limitation d'accélération 2
3.2 Frottements 2
3.3 Saturation moteur 2
3.4 Bruit moteur 3
3.5 Cinématique finale 3
3.6 Travail demandé 3

1 Objectifs du TP

2 Contexte

3 Simulation réaliste d'un moteur différentiel

Jusqu'à présent, nous avons utilisé un modèle cinématique **idéal** : les vitesses changent instantanément et le robot obéit parfaitement aux commandes.

Dans la réalité, ce n'est jamais le cas.

Un robot réel possède :

— une masse,
— une inertie,
— des limites moteur,
— des frottements,
— des imprécisions mécaniques.

Pour simuler un robot plus réaliste, nous allons enrichir notre modèle.

3.1 Limitation d'accélération

Un moteur réel ne peut pas passer instantanément d'une vitesse à une autre.
On introduit donc une **accélération maximale** :

$$a_{\max}$$

La variation de vitesse linéaire est limitée par :

$$v_{k+1} = v_k + \text{clip}(v_{cmd} - v_k,\ -a_{\max}\Delta t,\ a_{\max}\Delta t)$$

Même principe pour la rotation :

$$\omega_{k+1} = \omega_k + \text{clip}(\omega_{cmd} - \omega_k,\ -\alpha_{\max}\Delta t,\ \alpha_{\max}\Delta t)$$

où :
— v_{cmd} = vitesse demandée
— ω_{cmd} = rotation demandée
— $\alpha_{\max}$ = accélération linéaire/angulaire maximale

Interprétation physique : Le robot met un certain temps à atteindre la vitesse demandée.

3.2 Frottements

Un robot réel ralentit naturellement même sans commande.
On modélise cela par une décroissance proportionnelle :

$$v \leftarrow v(1 - \lambda\Delta t)$$

$$\omega \leftarrow \omega(1 - \lambda\Delta t)$$

où λ est le coefficient de frottement.

Effet :
— plus λ est grand -> robot s'arrête vite
— plus λ est petit -> robot glisse longtemps

3.3 Saturation moteur

Les moteurs ont une vitesse maximale :

$$|v| \leq v_{\max}$$

$$|\omega| \leq \omega_{\max}$$

On applique donc :

$$v = \text{clip}(v,\ -v_{\max},\ v_{\max})$$

$$\omega = \text{clip}(\omega,\ -\omega_{\max},\ \omega_{\max})$$

Interprétation : Même si l'on demande une vitesse très grande, le robot ne peut pas aller plus vite que ses capacités physiques.

3.4 Bruit moteur

Les moteurs réels ne sont jamais parfaitement précis.
On ajoute un bruit aléatoire :

$$v \leftarrow v + \varepsilon_v$$

$$\omega \leftarrow \omega + \varepsilon_\omega$$

avec :

$$\varepsilon \sim \mathcal{U}(-\sigma, \sigma)$$

ε est une variable aléatoire tirée selon une loi uniforme entre σ et $+\sigma$

Voir `random.uniform` en python.

Effet :
— trajectoire légèrement irrégulière
— simulation plus réaliste

3.5 Cinématique finale

Une fois les vitesses mises à jour, on calcule la nouvelle pose :

$$\theta_{k+1} = \theta_k + \omega \Delta t$$

$$x_{k+1} = x_k + v \cos(\theta_k) \Delta t$$

$$y_{k+1} = y_k + v \sin(\theta_k) \Delta t$$

3.6 Travail demandé

Implémenter une classe :

`MoteurDifferentielRealiste`

qui :

— stocke les vitesses réelles
— applique les limites d'accélération
— applique les frottements
— applique les saturations
— ajoute du bruit
— met à jour la position du robot

Le robot ne doit pas connaître ces calculs. Ils doivent être entièrement gérés par le moteur.

Procéder étape par étape : (1) implémenter seulement l'accélération ; (2) ajouter les frottements ; (3) ajouter les saturations ; (4) ajouter le bruit. Tester chaque étape avant de passer à la suivante.